



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment 1

Student Name: Ashutosh Yadav

UID: 23BCS11023

Branch: BE-CSE

Section: Krg-3-A

Semester: 6

Date of performance: 08-01-2026

Subject name: System Design

Subject code: 23CSH-314

Aim

To study and design a scalable URL Shortener system by analyzing its functional requirements, non-functional requirements, API design, database schema, and low-level design approaches including counter-based short URL generation for distributed systems.

Objective

1. To understand the working of a URL shortening service.
2. To design REST APIs for URL creation and redirection.
3. To identify suitable database and server choices.
4. To analyze different short URL generation techniques.
5. To understand scalability challenges and their solutions.

Problem Statement

Design a URL Shortener system that converts long URLs into short URLs and redirects users from short URLs to original long URLs with **low latency, high availability, and scalability**.

Requirements

A. Functional Requirements

1. Generate a short URL from a long URL.
2. Support custom URLs (optional).
3. Support URL expiration (default and custom).
4. Redirect short URL to the original long URL.

B. Non-Functional Requirements



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

1. **Low Latency:** URL creation and redirection < 200 ms.
2. **Scalability:**
 - 100M daily active users
 - 1B shortened URLs
3. **Availability:** System should be available 24×7.
4. **Uniqueness:** Each short URL must be unique.
5. **CAP Theorem:**
 - Availability > Consistency
 - Eventual Consistency model.

API Design

1. Create Short URL

Endpoint: POST /v1/url

Request Body:

```
{  
  "longURL": "string", "expirationDate": "date"  
}
```

Response:

```
{  
  "shortURL": "https://short.ly/2bi"  
}
```

2. Redirect Short URL

Endpoint: GET /v1/url/{shortURL}

Action: Redirects to original long URL.

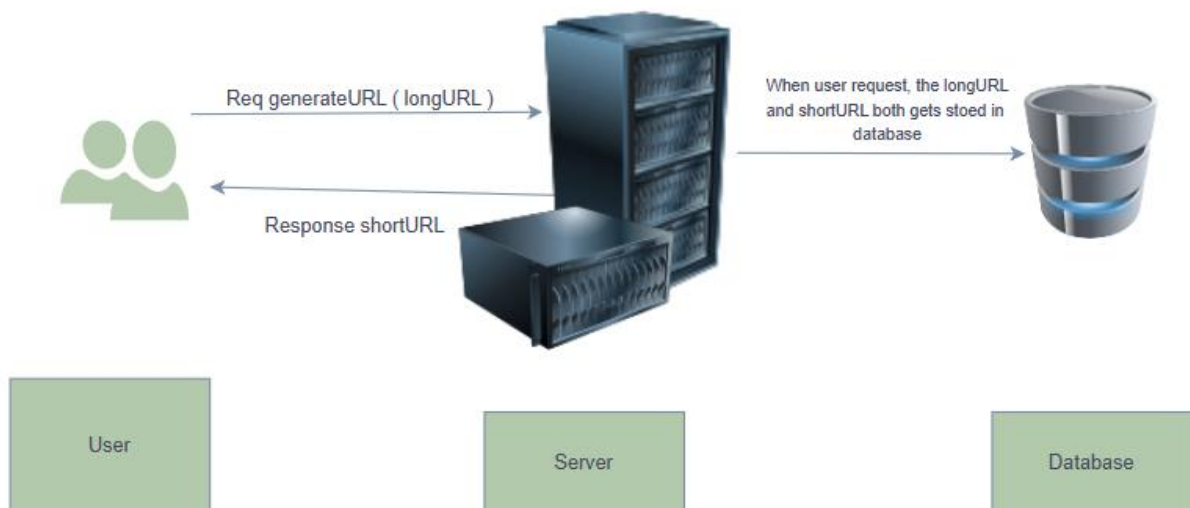
Database Design

URL Table

Column	Description
id	Primary Key
shortURL	Unique
longURL	Original URL
customURL	Optional
expirationDate	Expiry Time

Low Level Design (LLD)

Approach 01: Hashing (Encryption)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Method:

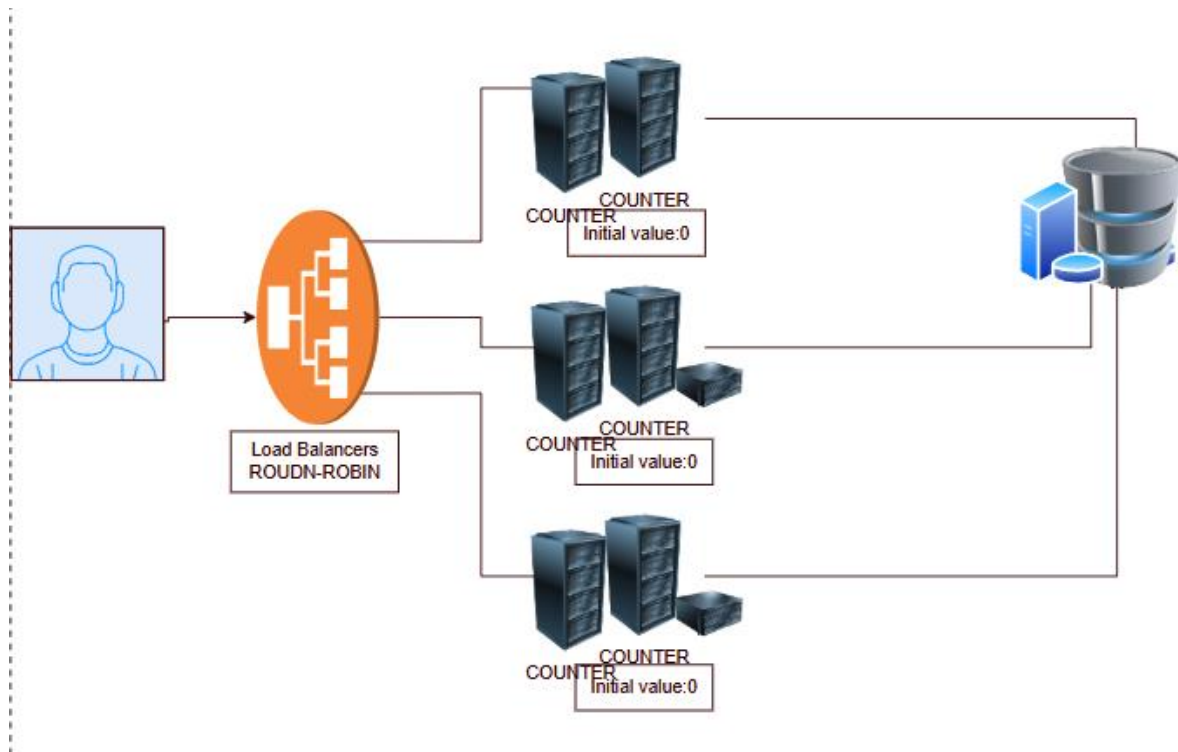
$\text{shortURL} = \text{encrypt}(\text{longURL})$

Algorithms: MD5, SHA-1, Base64

Problems

1. Long hash length (not short).
2. High collision probability.
3. Multiple DB lookups.
4. High latency at scale.

Not suitable for large-scale systems

Approach 02: Counter-Based Approach (Recommended)**Logic:**

1. Generate a global counter value.
2. Convert counter to Base62.
3. Use Base62 value as short URL.

Example:

Counter = 10000



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Base62 = 2bi

Short URL = short.ly/2bi

✓ Advantages

- No collision
- Fast generation
- Predictable length

Scaling Challenges & Solutions

Problem 1: Monolithic Server

- Cannot handle 100M users.

Solution: Horizontal Scaling (Multiple Servers)

Problem 2: Distributed Counter Conflict

- Multiple servers may generate same counter.

Solution:

Use **Global Counter in Cache (Redis)** with atomic increment.

Conclusion

The counter-based approach with centralized atomic counters provides the most efficient, scalable, and collision-free solution for a URL Shortener system handling millions of users with low latency.